

Algebraic data types, Data Oriented Programming and Pattern Matching in C#

Praseed Pai K.T.

Algebraic Data Types (Some theory!)

- Product Types (aka Records)
- Sum Types (aka Union types)
- Exponential Types (aka Function Types)
- Adhoc Polymorphism using Sum,Product,Exponential Types
- Data Oriented Programming
- Switch Expressions
- Pattern Matching

Sum Types

- Represent a value that can be **one of several types**.
- Common in functional languages like F#, Rust, and Haskell.
- In C# 10, you emulate this using **abstract records** and **pattern matching**.

```
public abstract record Expr;
```

```
public record Constant(double Value) : Expr;
```

```
public record Add(Expr Left, Expr Right) : Expr;
```

- Evaluated using **switch expressions** and **type patterns**.

Product Types

- Represent a value that combines multiple fields.
- like a tuple or struct — each instance holds multiple values.

```
public record Person(string Name, int Age);
```

Exponential Types (aka Function Types)

Represent a mapping from one type to another: $A \rightarrow B$.

In category theory, this is analogous to exponentiation: B^A .

In C#, this is expressed via delegates or lambdas:

```
Func<int, string> describe = age => $"Age: {age}";
```

How these are combined together?

- C# 10 brings these concepts closer to functional programming:
- Sum types via abstract records + pattern matching
- Product types via concise record syntax
- Exponential types via lambdas, delegates, and `Func<>`
- Switch expressions unify control flow with pattern matching

What about .NET 9 and before?

- Sum Types can be implemented using OneOf library

```
////////////////////////////////////
```

```
//
```

```
// A Simple Shape Hierarchy
```

```
public record Circle(double rad);
```

```
public record Rectangle(double l , double b );
```

```
public record Triangle(double b , double h );
```

```
public class Shape : OneOfBase<Circle,Rectangle,Triangle> {
```

```
    public Shape( OneOf<Circle,Rectangle,Triangle> inp ) : base(inp){}
```

```
}
```

Goodbye Composite/Visitor

- Algebraic data types help you to model Types
- Processing is done through Recursive function, Functional switch
- No complicated manouever like Double Dispatch etc
- Enables Adhoc polymorphism

Composite/Visitor version of Evaluator

```
class Constant : Expr {
    public double Value;
    public override double Evaluate() => Value;
}
class Add : Expr {
    public Expr Left, Right;
    public override double Evaluate() => Left.Evaluate() + Right.Evaluate();
}
interface IVisitor {
    double Visit(Constant c);
    double Visit(Add a);
}
abstract class Expr {
    public abstract double Accept(IVisitor visitor);
}
```

Algebraic data type version of Evaluator

```
public abstract record Expr;  
public record Constant(double Value) : Expr;  
public record Add(Expr Left, Expr Right) : Expr;  
  
double Evaluate(Expr expr) => expr switch  
{  
    Constant c => c.Value,  
    Add a => Evaluate(a.Left) + Evaluate(a.Right),  
    _ => throw new InvalidOperationException()  
};
```

Four Principles of Data Oriented Programming

- Separate Code from Data
- Represent Data Types with Generic structures (Records, Unions, List, Tuple)
- Use data validation as a Separate Concern
- Model state with Immutable Data

Questions?

- If any!